From Zero to RISC-V: A Semester Roadmap

Goal: Learn Verilog from scratch → build a working 8-bit CPU (Weeks 1–3) → extend into a real RV32I RISC-V core running on FPGA (rest of semester).

Phase 0: Setup (Days 1–2)

- [ ] Install **Icarus Verilog** + **GTKWave** (free, works on Windows/Linux/Mac) — this is all you need for simulation, no FPGA required yet
- [ ] Bookmark **EDA Playground** (edaplayground.com) — browser-based Verilog simulator, great for quick tests without local install
- [ ] Install **Xilinx Vivado WebPACK** (free) — only needed once you're ready to target actual hardware (Basys 3). Skip for now if just simulating.
- [ ] Set up a GitHub repo for the project now — commit as you go. This becomes part of your resume/portfolio link.

Phase 1: Learn Verilog Fundamentals (Week 1)

Don't try to learn "all of Verilog" — learn just enough to build digital circuits.

Concepts to master, in order:

1. Modules, ports, wires vs regs
2. Combinational logic (`assign`, `always @(*)`)
3. Sequential logic (`always @(posedge clk)`) — this is the big mental shift from software
4. Basic building blocks: multiplexers, decoders, adders, comparators
5. Finite State Machines (FSMs) — critical, since your CPU control unit is an FSM
6. Testbenches — how to simulate and verify a module without hardware

Mini-exercises to build this week (each ~1–2 hrs):

- [ ] 4-bit adder with carry
- [ ] 2-to-1 and 4-to-1 multiplexer
- [ ] D flip-flop, then an 8-bit register built from flip-flops
- [ ] Simple traffic light FSM (classic exercise, builds FSM intuition fast)
- [ ] A basic ALU: given two 8-bit inputs and a 3-bit opcode, output ADD/SUB/AND/OR/XOR result

Resources:

- Nandland's Verilog tutorials (nandland.com) — beginner-friendly, free
- ChipVerify Verilog tutorial — good reference-style docs

- HDLBits (hdlbits.01xz.net) — the best practice-problem site, does auto-grading in-browser, use this heavily

---

Phase 2: Design & Build an 8-bit CPU (Weeks 2–3)

This is your "moderate depth" milestone — a complete, demoable project even if nothing else gets built.

2.1 — Design the ISA (Instruction Set Architecture) first, on paper Keep it small. A classic minimal 8-bit ISA:

Instruction	Opcode (example)	Function
LOAD	0000	Load value from memory into register
STORE	0001	Store register value into memory
ADD	0010	Add two registers
SUB	0011	Subtract two registers
JUMP	0100	Unconditional jump
JZ	0101	Jump if zero flag set
OUT	0110	Output register value (to LEDs later)
HALT	1111	Stop execution

☐ Finalize your own instruction format (e.g., 4-bit opcode + 4-bit operand for a simple design)
☐ Write out 2–3 sample programs by hand in your ISA (e.g., "add two numbers and output result", "count from 0 to 10")

2.2 — Build the datapath modules

☐ Register file (a small set of general-purpose registers, e.g., 4×8-bit)
☐ ALU (reuse/extend from Phase 1 exercise)
☐ Program counter (PC) with increment + jump logic
☐ Instruction memory (ROM, can preload with your hand-assembled program)
☐ Data memory (RAM)

2.3 — Build the control unit

☐ FSM that sequences: Fetch → Decode → Execute (classic 3-stage sequencing, all in one clock-multi-cycle style is easiest for a first CPU)

- [ ] Wire the control signals to the datapath modules

2.4 — Integrate and verify

- [ ] Write a top-level `cpu.v` module tying everything together
- [ ] Write a testbench that loads a hand-assembled program and checks the outputs step-by-step in GTKWave
- [ ] **Milestone: your hand-written test program executes correctly in simulation** 🎉

2.5 — (Optional but recommended) Deploy to FPGA

- [ ] If you have a Basys 3 or similar, map CPU outputs to onboard LEDs/7-segment display so you can literally watch it compute
- [ ] This is a great demo moment — showing a professor or interviewer a physical board blinking out the result of your CPU's computation is memorable

Checkpoint — by end of Week 3 you should have: A working 8-bit CPU, simulated (and ideally FPGA-deployed), with your own custom ISA, a few test programs, and a GitHub repo documenting it. This alone is a legitimate, complete resume project if the semester timeline gets tight later.

Phase 3: Bridge to RISC-V (Weeks 4–5)

Before jumping into RV32I, understand *why* RISC-V is designed the way it is — this makes the next phase much easier.

- [ ] Read the RISC-V Green Card / RV32I base instruction set reference (free PDF, widely available)
- [ ] Understand RISC-V instruction formats: R-type, I-type, S-type, B-type, U-type, J-type
- [ ] Understand the register set: 32 general-purpose registers (x0–x31), x0 hardwired to zero
- [ ] Compare it mentally to your Phase 2 custom ISA — you'll notice RISC-V is really just a more rigorous, standardized version of what you already built

Why this matters for your resume: anyone can invent a toy ISA; implementing an *industry-standard* ISA (used by real chips at SiFive, Western Digital, and others) is what makes this project credible to an interviewer.

Phase 4: Build a Single-Cycle RV32I Core (Weeks 6–10)

This is the heart of the semester project.

4.1 — Scope your instruction subset first Don't try to implement all of RV32I at once. A solid, achievable subset:

- Arithmetic: `add`, `sub`, `and`, `or`, `xor`, `slt`

- Immediate versions: `addi`, `andi`, `ori`
- Memory: `lw`, `sw`
- Branches: `beq`, `bne`
- Jump: `jal`
- (Add more later if time allows: `slli`, `srli`, `lui`, etc.)

4.2 — Build module by module

☐ Register file (32×32-bit, x0 hardwired to 0)
☐ Instruction decoder (extract opcode, funct3/funct7, rs1, rs2, rd, immediate — with correct sign extension per instruction type)
☐ ALU (extend your Phase 2 ALU to 32-bit, add SLT logic)
☐ Control unit (this time it can be **combinational**, not an FSM, since single-cycle executes one instruction per clock — this is a nice conceptual step up)
☐ Instruction memory + data memory
☐ PC logic with branch/jump support

4.3 — Integrate and verify

☐ Assemble small test programs using a RISC-V assembler (or hand-assemble a few instructions first to sanity-check)
☐ Testbench: load program, run, check register/memory values against expected results
☐ **Milestone: single-cycle RV32I core correctly runs a hand-written test program** 🎉

4.4 — Deploy to FPGA

☐ Get it running on real hardware, not just simulation
☐ Add memory-mapped I/O: map an address range to switches (input) and LEDs (output) so the core can interact with the physical board

Resources for this phase:

- "RISC-V Reader" (free open textbook) or the official RISC-V ISA spec (unprivileged spec PDF)
- OneLoneCoder / nandland-style single-cycle RISC-V build videos exist on YouTube — useful for unsticking, but build your own, don't just copy
- HDLBits has some RISC-V-adjacent datapath problems if you want extra practice

---

Phase 5: Toolchain Integration (Weeks 11–12) — makes the project feel "real"

☐ Install the RISC-V GNU toolchain (`riscv-gnu-toolchain`) or use an online compiler
☐ Compile a **tiny C program** (e.g., blink an LED pattern, compute Fibonacci) down to RV32I machine code

- [ ] Load the compiled machine code into your instruction memory and run it on your core
- [ ] **Milestone: your CPU runs actual compiled C code, not just hand-written assembly** — this is a huge credibility jump

---

Phase 6 (Stretch Goal, Weeks 13–15): Add a Pipeline

If time allows, converting single-cycle → a simple 2 or 3-stage pipeline teaches hazard detection, forwarding, and stalling — core computer architecture concepts that show up in interviews constantly.

- [ ] Split into stages (e.g., Fetch/Decode, Execute/Memory/Writeback)
- [ ] Add pipeline registers between stages
- [ ] Identify and handle data hazards (forwarding or stalling)
- [ ] Identify and handle control hazards (branch handling — even a simple "flush on branch" is fine for a first pipeline)

This is genuinely hard and it's fine if you don't finish it — even attempting it and documenting what you learned about hazards is valuable to talk about in interviews.

Final Deliverables Checklist (for resume + demo)

- [ ] GitHub repo with clean commit history, README explaining the architecture
- [ ] Block diagram of your CPU/core (draw.io or similar — visuals matter for demos)
- [ ] A short writeup: ISA design decisions, what you'd do differently, what you learned
- [ ] Working FPGA demo video or live demo (LEDs blinking out a real computation — very effective in interviews)
- [ ] Resume line, e.g.: *"Designed and implemented a single-cycle RV32I RISC-V processor core in Verilog from scratch; verified via testbench simulation and deployed on Xilinx Artix-7 FPGA; compiled and executed C programs on the custom core using the RISC-V GNU toolchain."*

Suggested Pacing Summary

Weeks	Focus	Outcome
0	Setup	Tools installed
1	Verilog fundamentals	Comfortable writing/simulating basic circuits
2–3	8-bit CPU	**Milestone 1**: working custom CPU, simulated (+ FPGA if possible)
4–5	RISC-V theory	Understand RV32I formats and design
6–10	Single-cycle RV32I core	**Milestone 2**: real RISC-V core running programs
11–12	Toolchain integration	**Milestone 3**: compiled C code runs on your CPU
13–15	Pipeline (stretch)	Optional but high-value if time allows